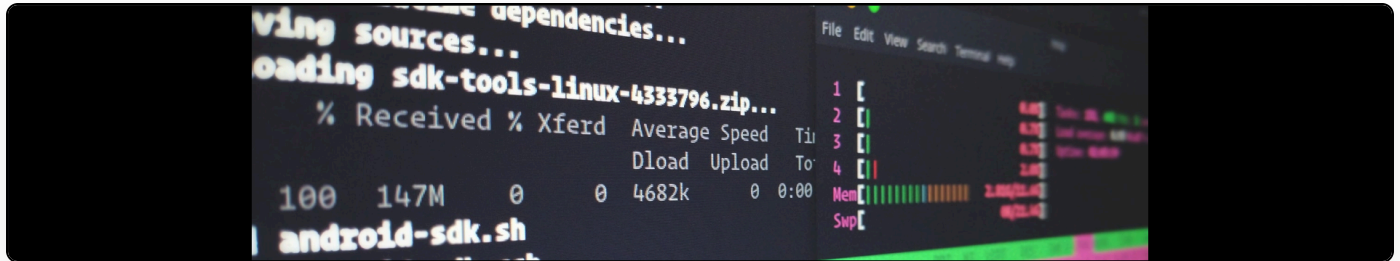


# C# DataExecutor class available

**Cite as:** Martin Paul Eve, "C# DataExecutor class available", *eve.gd: Martin Paul Eve*, June 26, 2007, <https://doi.org/10.59348/gepnb-vc464>.

---



One of the questions I see most frequently on Freenode's ##csharp irc channel is how to use a MySQL Database in .NET. I've therefore provided the class that I use for basic database operations. You can find it at <http://www.martineve.com/2007/06/25/c-mysql-dataexecutor-class/>.

The class supports both strongly and weakly typed datasets and usage is as follows:

1. Get the MySql Connector.NET
2. Reference it in your project
3. Use the following for strongly typed datasets:

</ul>

```

MySQL.Data.MySqlClient.MySqlCommand cmd = new
MySQL.Data.MySqlClient.MySqlCommand("SELECT * FROM table WHERE blah=@blah");
cmd.Parameters.Add("@blah", "test");

Tools.DataExecutor de = null;

try{

    de = new Tools.DataExecutor(cmd, false);

    DataSchemas.app.tableDataTable test = new DataSchemas.app.tableDataTable();
    de.DataSetSchema(test);

    DataSchemas.app.tableRow row = null;

    if (test.Rows.Count > 0)
    {
        row = (DataSchemas.app.tableRow)test.Rows[0];
    }

} finally {
    de.Close();
}

```

and this for weakly typed datasets:

```

MySQL.Data.MySqlClient.MySqlCommand cmd = new
MySQL.Data.MySqlClient.MySqlCommand("SELECT * FROM table WHERE blah=@blah");
cmd.Parameters.Add("@blah", "test");

Tools.DataExecutor de = new Tools.DataExecutor(cmd, false);

DataSet ds = new DataSet();

de.Adapter.Fill(ds);

```

Easy eh? :)

The parameterization of the queries will protect you against SQL Injection exploits so I'd recommend you do that at all times. Your connection string should go in your Web.Config file and then be specified inside the DataExecutor class. It's worth pointing out that if you don't set

oldsyntax=true; then you must ensure that you use ? for your parameter names as opposed to @.